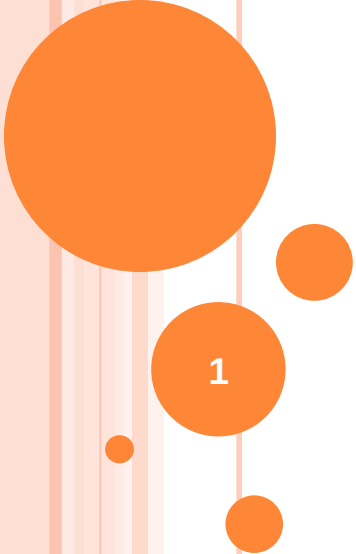




AN OVERVIEW OF PYTHON

1

WHAT IS OOP?

- ❑ OOP is a powerful way to approach the task of programming.
- ❑ OOP encourages developers to decompose a problem into its constituent parts.
- ❑ Each component becomes a self-contained object that contains its own instructions and data that relate to that object.
- ❑ So, complexity is reduced and the programmer can manage larger programs.

WHAT IS OOP? (CONT.)

- All OOP languages, including Python, share three common defining traits:
 - Encapsulation
 - Binds together code and data
 - Polymorphism
 - Allows one interface, multiple methods
 - Inheritance
 - Provides hierarchical classification
 - Permits reuse of common code and data

PYTHON CONSOLE I/O (OUTPUT)

```
print("Hello, World!")
```

```
name = "Alice"
```

```
print("Hello,", name)
```

Output:

Hello, World!

Hello, Alice

PYTHON CONSOLE I/O (OUTPUT)

Formatted String:

```
age = 30
```

```
print(f"Name: {name}, Age: {age}")
```

Output: Name: Alice, Age: 30

CONSOLE I/O (INPUT)

```
print("Enter your name:")  
name = input()  
print(f"You entered: {name}")
```

```
user_input = input("Enter your name: ")  
print("You entered:", user_input)
```

CONSOLE I/O (INPUT)

Integer Input:

```
age_str = input("Enter your age: ")  
age_int = int(age_str)  
print(f"Next year you will be {age_int + 1}.")
```

PYTHON COMMENTS

- ❑ Multi-line comments

```
''''
```

This text is a string literal that acts as a comment

because it is not assigned to a variable.

```
''''
```

- ❑ Single line comments

This entire line is a comment

CLASSES: A FIRST LOOK

- General syntax -

```
class class-name:  
    // private and public variables  
  
    // private and public functions
```

CLASSES: A FIRST LOOK (CONT.)

- A class declaration is a logical abstraction that defines a new type.
- It determines what an object of that type will look like.
- An object declaration creates a physical entity of that type.
- That is, an object occupies memory space, but a type definition does not.

CLASSES: A FIRST LOOK (CONT.)

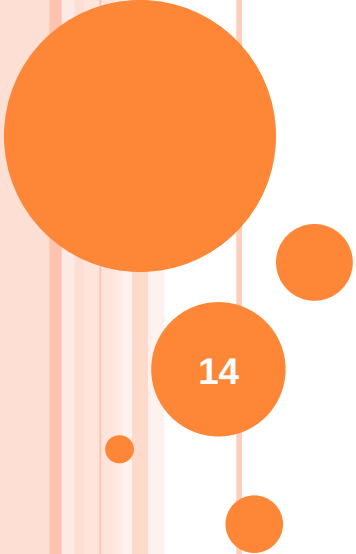
- Each object of a class has its own copy of every variable declared within the class (except static variables which will be introduced later), but they all share the same copy of member functions.
 - How do member functions know on which object they have to work on?
 - The answer will be clear when “*self*” pointer is introduced

POLYMORPHISM

- Polymorphism refers to functions with the same name that can be executed on many objects or classes.
- Runtime Polymorphism:
 - Functions execution of same name is decided in runtime.
- Compile-time Polymorphism:
 - Functions execution of same name is decided in compile time.

POLYMORPHISM

- Runtime Polymorphism:
 - Function overriding
- Compile-time Polymorphism:
 - Function overloading
 - Python does not support compile time polymorphism



INTRODUCING CLASSES

14

CONSTRUCTORS

- Every object we create will require some sort of initialization.
- A class's constructor is automatically called by the compiler each time an object of that class is created.
- A constructor function has the **same name** as the class and has **no return type** in most of OOP languages.
- There is no explicit way to call the constructor.

CONSTRUCTORS

- Python is a little different
- it has a constructor and an initializer.
- The constructor function is rarely used.
- constructor (`__new__`) creates the object instance
- while the initializer (`__init__`) sets the initial state or attributes of that newly created object.

DESTRUCTORS

- The complement of a constructor is the destructor.
- This function is automatically called by the compiler when an object is destroyed.
- The name of a destructor is the *name of its class*, preceded by a ~ in most of the OOP languages.
- There is explicit way to call the destructor but highly discouraged.

DESTRUCTORS

- In Python, the destructor is a special method named `__del__()` that is called just before an object is destroyed or garbage-collected
- Unlike in languages like C++, destructors are not frequently needed in Python because its automatic garbage collector handles memory management.

CONSTRUCTORS THAT TAKE PARAMETERS

- In Python we will generally use initializer to initialize the object.
- Initializer can be without any parameters
- And Initializer can take parameter as well.

OBJECT POINTERS

- In Python, `self` is a parameter that acts as a reference to the current instance of a class
- In C++, 'this' pointer is an implicit pointer available inside all non-static member function

INHERITANCE

- ❑ Inheritance is a core principle where a new class (subclass/child) acquires properties (attributes) and behaviors (methods) from an existing class (superclass/parent).

TYPES OF INHERITANCE IN PYTHON

- ❑ **Single Inheritance:** A child class inherits from only one parent class.
- ❑ **Multiple Inheritance:** A class can inherit from more than one parent class.
- ❑ **Multilevel Inheritance:** A chain of inheritance where a class inherits from a derived class, creating a grandparent -parent-child relationship.